READ DATA

```
import os
import random
from google.colab import drive

drive.mount('/content/drive')
train_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/train/PNEUMONIA'
test_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/test/PNEUMONIA'
train_normal_dir = '/content/drive/MyDrive/TA/Dataset/train/NORMAL'
test_normal_dir = '/content/drive/MyDrive/TA/Dataset/test/NORMAL'
```

```
Mounted at /content/drive
```

```
def get_all_files(dirs, allowed_extensions=['.jpg', '.jpeg', '.png']):
    all_files = []
    for dir_path in dirs:
        files = [os.path.join(dir_path, f) for f in os.listdir(dir_path)
                 if not f.startswith('.') and os.path.splitext(f)[1].lower() in allowed_extensions]
        all_files.extend(files)
    return all_files

def random_select_files(file_list, subset_size):
    if subset_size > len(file_list):
        raise ValueError("Jumlah file yang diminta lebih besar daripada jumlah file yang tersedia.")
    return random.sample(file_list, subset_size)

def count_files(directory):
  return len(os.listdir(directory))


all_pneumonia_files = get_all_files([train_pneumonia_dir, test_pneumonia_dir])
all_normal_files = get_all_files([train_normal_dir, test_normal_dir])

print(f"train pneumon : {count_files(train_pneumonia_dir)}")
print(f"test pneumon : {count_files(test_pneumonia_dir)}")
print(f"train norm : {count_files(train_normal_dir)}")
print(f"test norm : {count_files(test_normal_dir)}")
```

```
print(f"pneumon : {len(all_pneumonia_files)}")
print(f"norm : {len(all_normal_files)}")
```

```
train pneumon : 3875
test pneumon : 390
train norm : 1341
test norm : 234
pneumon : 4265
norm : 1575
```

```
train_pneumonia_samples = random_select_files(all_pneumonia_files, 1181)
test_pneumonia_samples = random_select_files([f for f in all_pneumonia_files if f not in train_pneumonia_samples], 394)

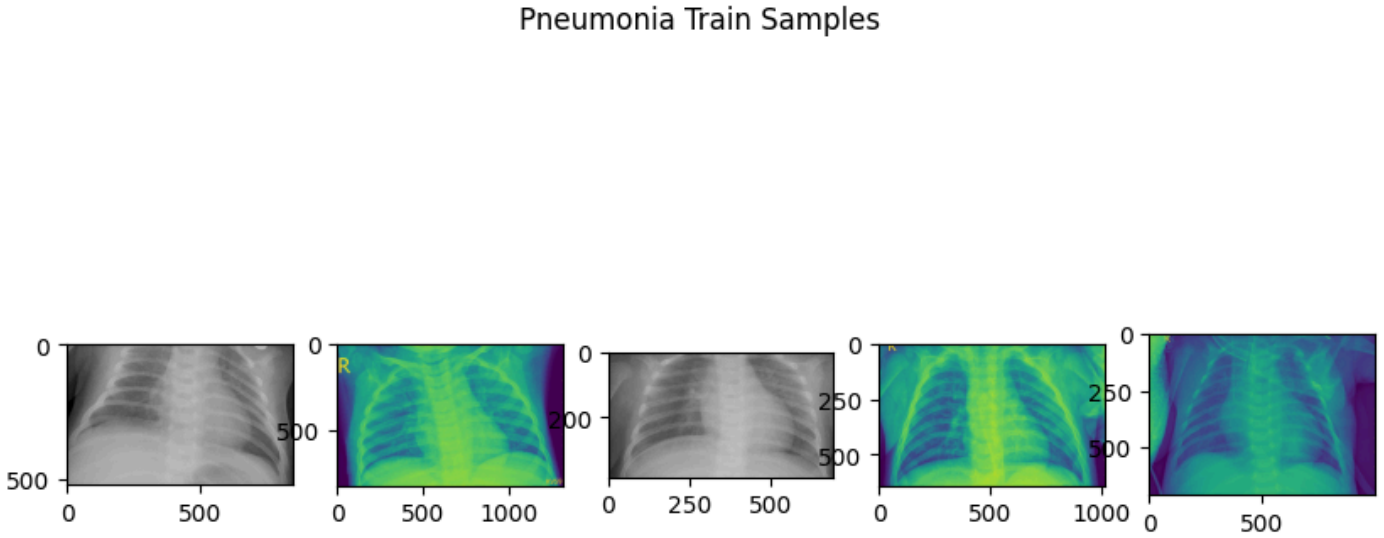
train_normal_samples = random_select_files(all_normal_files, 1181)
test_normal_samples = random_select_files([f for f in all_normal_files if f not in train_normal_samples], 394)

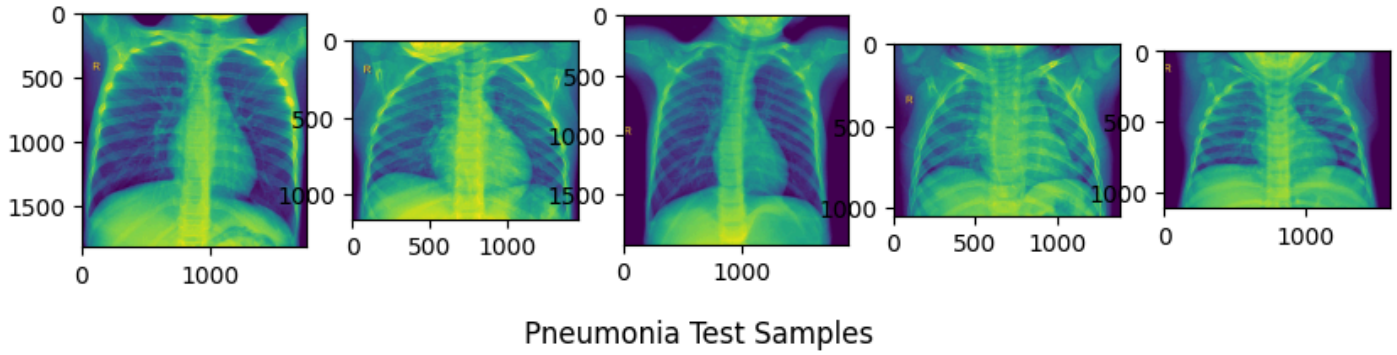
#train_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in train_pneumonia_files]
#test_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in test_pneumonia_files]
#train_normal_samples = [os.path.join(train_normal_dir, file) for file in train_normal_files]
#test_normal_samples = [os.path.join(train_normal_dir, file) for file in test_normal_files]
```

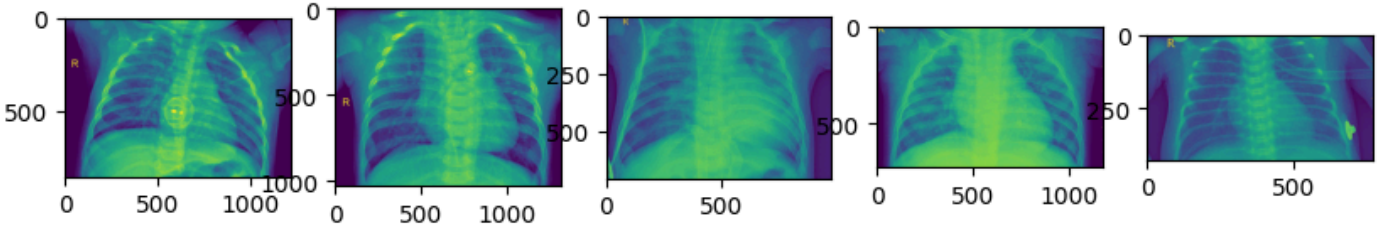
```
from PIL import Image
import matplotlib.pyplot as plt

def show_sample_images(sample_files,ammount, title):
    plt.figure(figsize=(10, 5))
    plt.suptitle(title)
    for i in range(ammount):
        plt.subplot(1, ammount, i+1)
        img = Image.open(sample_files[i])
        plt.imshow(img)
    plt.show()

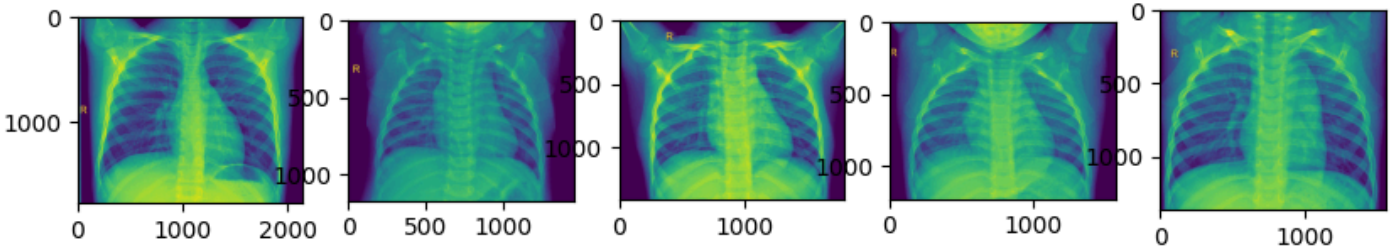
show_sample_images(train_pneumonia_samples, 5, 'Pneumonia Train Samples')
show_sample_images(train_normal_samples, 5, 'Normal Train Samples')
show_sample_images(test_pneumonia_samples, 5, 'Pneumonia Test Samples')
show_sample_images(test_normal_samples, 5, 'Normal Test Samples')
```


Pneumonia Train Samples
Normal Train Samples


Pneumonia Test Samples



Normal Test Samples



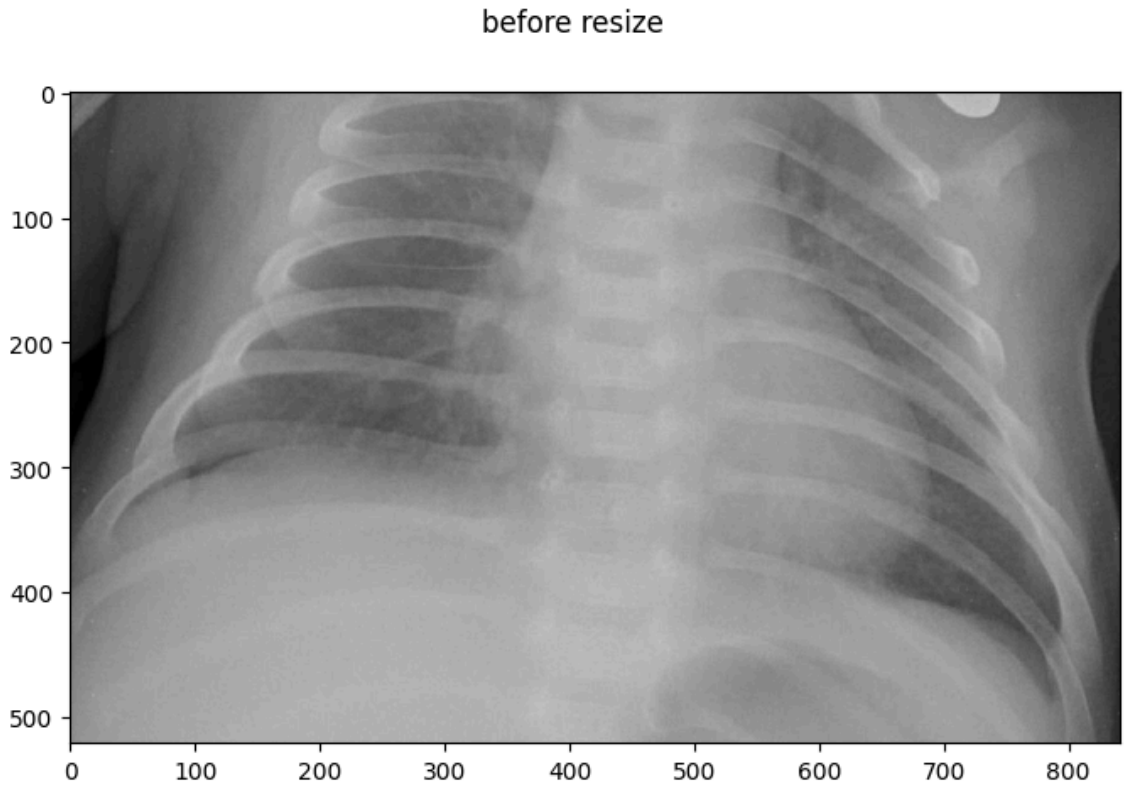
RESIZE

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

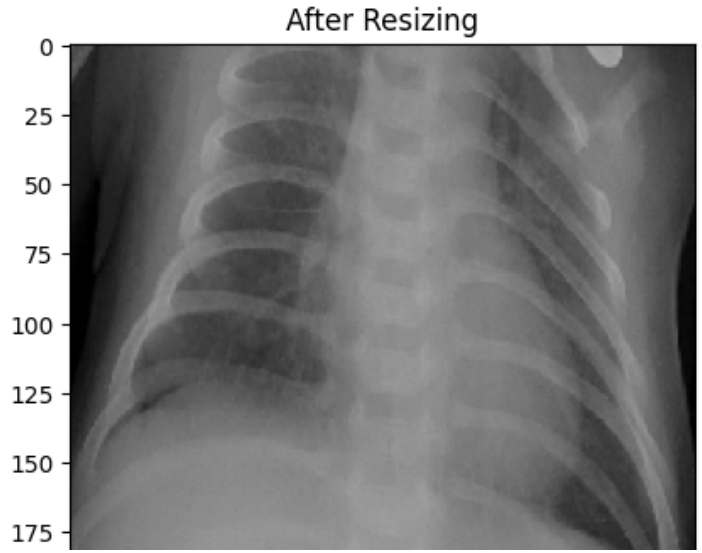
def load_images(file_paths, target_size=(224, 224)):
    images = []
    for i, file_path in enumerate(file_paths):
        img = load_img(file_path, target_size=target_size)
        img_array = img_to_array(img)
        img_array = img_array - 30
        img_array = img_array / 255.0
        images.append(img_array)
    return images

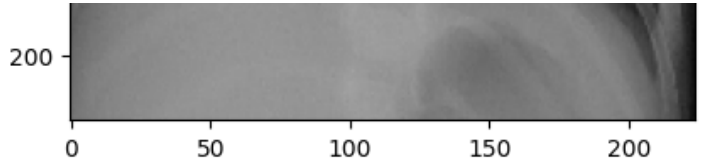
train_pneumonia_images = load_images(train_pneumonia_samples)
test_pneumonia_images = load_images(test_pneumonia_samples)
train_normal_images = load_images(train_normal_samples)
test_normal_images = load_images(test_normal_samples)


show_sample_images(train_pneumonia_samples, 1, 'before resize')
plt.imshow(train_pneumonia_images[0])
plt.title('After Resizing')
```




```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.1176
Text(0.5, 1.0, 'After Resizing')
```





GLCM

```
import numpy as np
import cv2
from skimage.feature import graycomatrix, graycoprops

def calculate_glcm(image):
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    distances = [1, 2, 3]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]

    glcm = graycomatrix(gray_image,
                        distances=distances,
                        angles=angles,
                        levels=256,
                        symmetric=True,
                        normed=True)

    properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy', 'correlation', 'ASM']
    features = {}

    for prop in properties:
        values = graycoprops(glcm, prop).flatten()
        features[f'{prop}_mean'] = np.mean(values)
        features[f'{prop}_std'] = np.std(values)
        features[f'{prop}_var'] = np.var(values)

    glcm_entropy = -np.sum(glcm * np.log2(glcm + 1e-10))
    features['entropy'] = glcm_entropy

    return features
```

```
import cv2

def extract_glcm_features(image_list):
    glcm_features_list = []
    for image in image_list:
        image_uint8 = (image * 255).astype(np.uint8)
        features = calculate_glcm(image_uint8)
        glcm_features_list.append(features)
    return glcm_features_list

train_pneumonia_glcm = extract_glcm_features(train_pneumonia_images)
test_pneumonia_glcm = extract_glcm_features(test_pneumonia_images)
train_normal_glcm = extract_glcm_features(train_normal_images)
test_normal_glcm = extract_glcm_features(test_normal_images)

print("GLCM features for the first pneumonia training image:")
train_pneumonia_glcm[0]
```

```
GLCM features for the first pneumonia training image:
{'contrast_mean': np.float64(206.33296342260556),
 'contrast_std': np.float64(81.40201553427462),
 'contrast_var': np.float64(6626.288133042287),
 'dissimilarity_mean': np.float64(5.416154731848128),
 'dissimilarity_std': np.float64(1.4661402533671697),
 'dissimilarity_var': np.float64(2.1495672425435486),
 'homogeneity_mean': np.float64(0.2441545647266974),
 'homogeneity_std': np.float64(0.0435991244515646),
 'homogeneity_var': np.float64(0.0019008836529430184),
 'energy_mean': np.float64(0.026352049331390306),
 'energy_std': np.float64(0.0023348339086821275),
 'energy_var': np.float64(5.4514493811318605e-06),
 'correlation_mean': np.float64(0.9120807638522169),
 'correlation_std': np.float64(0.03642594940204396),
 'correlation_var': np.float64(0.0013268497898402667),
 'ASM_mean': np.float64(0.0006998819533451599),
 'ASM_std': np.float64(0.00012473502431855482),
 'ASM_var': np.float64(1.5558826291750465e-08),
 'entropy': np.float64(135.98989534811324)}
```

```
#import pandas as pd
#from google.colab import files

#train_pneumonia_df = pd.DataFrame(train_pneumonia_glcm)
#train_pneumonia_df['label'] = 'PNEUMONIA'
```

```
#test_pneumonia_df = pd.DataFrame(test_pneumonia_glcm)
#test_pneumonia_df['label'] = 'PNEUMONIA'

#train_normal_df = pd.DataFrame(train_normal_glcm)
#train_normal_df['label'] = 'NORMAL'

#test_normal_df = pd.DataFrame(test_normal_glcm)
#test_normal_df['label'] = 'NORMAL'

#all_glcm_df = pd.concat([train_pneumonia_df, test_pneumonia_df, train_normal_df, test_normal_df], ignore_index=True)

#excel_filename = 'glcm_features.xlsx'
#all_glcm_df.to_excel(excel_filename, index=False)

#files.download(excel_filename)

#print(f"GLCM features saved to '{excel_filename}' and downloaded.")
```

CNN

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.regularizers import l2

# GLCM - DF
train_pneumonia_glcm_df = pd.DataFrame(train_pneumonia_glcm)
test_pneumonia_glcm_df = pd.DataFrame(test_pneumonia_glcm)
train_normal_glcm_df = pd.DataFrame(train_normal_glcm)
test_normal_glcm_df = pd.DataFrame(test_normal_glcm)

train_pneumonia_glcm_df['label'] = 1  # 1 for pneumonia
test_pneumonia_glcm_df['label'] = 1
train_normal_glcm_df['label'] = 0  # 0 for normal
test_normal_glcm_df['label'] = 0

train_glcm_df = pd.concat([train_pneumonia_glcm_df, train_normal_glcm_df], ignore_index=True)
test_glcm_df = pd.concat([test_pneumonia_glcm_df, test_normal_glcm_df], ignore_index=True)
```

```
# Separate features
X_train = train_glcm_df.drop('label', axis=1)
y_train = train_glcm_df['label']
X_test = test_glcm_df.drop('label', axis=1)
y_test = test_glcm_df['label']

# Normal
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Model
model = Sequential([
    Dense(128, activation='relu', kernel_regularizer=l2(0.0001), input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

optimizer = Adam(learning_rate=0.001)
class_weights = {0: 1.5, 1: 0.8}

callbacks = [
    EarlyStopping(monitor='val_loss', patience=50, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=25, min_lr=1e-6)
]

model.compile(optimizer=optimizer,
              loss='binary_crossentropy',
              metrics=['accuracy', tf.keras.metrics.AUC()])

model.summary()

history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.3,
    callbacks=callbacks,
    class_weight=class_weights,
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. Whe
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	2,560
batch_normalization (BatchNormalization)	(None, 128)	512
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

```
 Total params: 3,201 (12.50 KB)
 Trainable params: 2,945 (11.50 KB)
 Non-trainable params: 256 (1.00 KB)
Epoch 1/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - accuracy: 0.6400 - auc: 0.7167 - loss: 0.7204 - val_accuracy: 0.7306 - val_auc: 0.0000e+00 - val_loss: 0.6726 -
Epoch 2/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8386 - auc: 0.9144 - loss: 0.3953 - val_accuracy: 0.6869 - val_auc: 0.0000e+00 - val_loss: 0.6908 -
Epoch 3/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8552 - auc: 0.9304 - loss: 0.3452 - val_accuracy: 0.6587 - val_auc: 0.0000e+00 - val_loss: 0.7056 -
Epoch 4/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8752 - auc: 0.9358 - loss: 0.3263 - val_accuracy: 0.7207 - val_auc: 0.0000e+00 - val_loss: 0.6521 -
Epoch 5/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8783 - auc: 0.9377 - loss: 0.3178 - val_accuracy: 0.7856 - val_auc: 0.0000e+00 - val_loss: 0.5738 -
Epoch 6/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8735 - auc: 0.9349 - loss: 0.3265 - val_accuracy: 0.8195 - val_auc: 0.0000e+00 - val_loss: 0.5217 -
Epoch 7/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8820 - auc: 0.9432 - loss: 0.3000 - val_accuracy: 0.8392 - val_auc: 0.0000e+00 - val_loss: 0.4862 -
Epoch 8/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8779 - auc: 0.9462 - loss: 0.2936 - val_accuracy: 0.8590 - val_auc: 0.0000e+00 - val_loss: 0.4207 -
Epoch 9/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8897 - auc: 0.9479 - loss: 0.2851 - val_accuracy: 0.8561 - val_auc: 0.0000e+00 - val_loss: 0.4463 -
Epoch 10/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8891 - auc: 0.9486 - loss: 0.2863 - val_accuracy: 0.8787 - val_auc: 0.0000e+00 - val_loss: 0.3810 -
Epoch 11/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8960 - auc: 0.9545 - loss: 0.2684 - val_accuracy: 0.8717 - val_auc: 0.0000e+00 - val_loss: 0.4182 -
Epoch 12/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9027 - auc: 0.9490 - loss: 0.2797 - val_accuracy: 0.8829 - val_auc: 0.0000e+00 - val_loss: 0.3657 -
Epoch 13/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8819 - auc: 0.9485 - loss: 0.2875 - val_accuracy: 0.8787 - val_auc: 0.0000e+00 - val_loss: 0.3888 -
Epoch 14/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9020 - auc: 0.9475 - loss: 0.2862 - val_accuracy: 0.8815 - val_auc: 0.0000e+00 - val_loss: 0.3468 -
Epoch 15/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8946 - auc: 0.9495 - loss: 0.2875 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3509 -
Epoch 16/100
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8903 - auc: 0.9447 - loss: 0.2985 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3599 -
Epoch 17/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8978 - auc: 0.9502 - loss: 0.2841 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3499 -
Epoch 18/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8897 - auc: 0.9512 - loss: 0.2819 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3773 -
Epoch 19/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9076 - auc: 0.9638 - loss: 0.2387 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3670 -
Epoch 20/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9074 - auc: 0.9592 - loss: 0.2513 - val_accuracy: 0.8829 - val_auc: 0.0000e+00 - val_loss: 0.3610 -
Epoch 21/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8935 - auc: 0.9495 - loss: 0.2939 - val_accuracy: 0.8801 - val_auc: 0.0000e+00 - val_loss: 0.3548 -
Epoch 22/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9065 - auc: 0.9678 - loss: 0.2284 - val_accuracy: 0.8872 - val_auc: 0.0000e+00 - val_loss: 0.3396 -
Epoch 23/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8972 - auc: 0.9494 - loss: 0.2828 - val_accuracy: 0.8731 - val_auc: 0.0000e+00 - val_loss: 0.3838 -
Epoch 24/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9038 - auc: 0.9547 - loss: 0.2695 - val_accuracy: 0.8872 - val_auc: 0.0000e+00 - val_loss: 0.3508 -
Epoch 25/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8930 - auc: 0.9483 - loss: 0.2914 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3519 -
Epoch 26/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9039 - auc: 0.9612 - loss: 0.2480 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3593 -
Epoch 27/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9111 - auc: 0.9639 - loss: 0.2479 - val_accuracy: 0.8801 - val_auc: 0.0000e+00 - val_loss: 0.3649 -
Epoch 28/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9084 - auc: 0.9598 - loss: 0.2494 - val_accuracy: 0.8872 - val_auc: 0.0000e+00 - val_loss: 0.3660 -
Epoch 29/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8934 - auc: 0.9606 - loss: 0.2476 - val_accuracy: 0.8872 - val_auc: 0.0000e+00 - val_loss: 0.3502 -
Epoch 30/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9098 - auc: 0.9561 - loss: 0.2654 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3582 -
Epoch 31/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8922 - auc: 0.9566 - loss: 0.2644 - val_accuracy: 0.8970 - val_auc: 0.0000e+00 - val_loss: 0.3291 -
Epoch 32/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9036 - auc: 0.9626 - loss: 0.2424 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3456 -
Epoch 33/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8972 - auc: 0.9584 - loss: 0.2570 - val_accuracy: 0.8829 - val_auc: 0.0000e+00 - val_loss: 0.3509 -
Epoch 34/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9019 - auc: 0.9556 - loss: 0.2619 - val_accuracy: 0.8942 - val_auc: 0.0000e+00 - val_loss: 0.3344 -
Epoch 35/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8860 - auc: 0.9551 - loss: 0.2702 - val_accuracy: 0.8801 - val_auc: 0.0000e+00 - val_loss: 0.3651 -
Epoch 36/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9071 - auc: 0.9650 - loss: 0.2344 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3622 -
Epoch 37/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9042 - auc: 0.9565 - loss: 0.2580 - val_accuracy: 0.8829 - val_auc: 0.0000e+00 - val_loss: 0.3880 -
Epoch 38/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9050 - auc: 0.9597 - loss: 0.2496 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3609 -
Epoch 39/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8989 - auc: 0.9602 - loss: 0.2519 - val_accuracy: 0.8829 - val_auc: 0.0000e+00 - val_loss: 0.3669 -
Epoch 40/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9143 - auc: 0.9709 - loss: 0.2118 - val_accuracy: 0.8914 - val_auc: 0.0000e+00 - val_loss: 0.3457 -
```

```
Epoch 41/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9043 - auc: 0.9620 - loss: 0.2423 - val_accuracy: 0.8815 - val_auc: 0.0000e+00 - val_loss: 0.3572 -
Epoch 42/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9095 - auc: 0.9620 - loss: 0.2470 - val_accuracy: 0.8815 - val_auc: 0.0000e+00 - val_loss: 0.3716 -
Epoch 43/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9138 - auc: 0.9681 - loss: 0.2241 - val_accuracy: 0.8829 - val_auc: 0.0000e+00 - val_loss: 0.3696 -
Epoch 44/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8960 - auc: 0.9599 - loss: 0.2561 - val_accuracy: 0.8886 - val_auc: 0.0000e+00 - val_loss: 0.3607 -
Epoch 45/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9049 - auc: 0.9616 - loss: 0.2399 - val_accuracy: 0.8956 - val_auc: 0.0000e+00 - val_loss: 0.3384 -
Epoch 46/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9005 - auc: 0.9643 - loss: 0.2376 - val_accuracy: 0.8815 - val_auc: 0.0000e+00 - val_loss: 0.3720 -
Epoch 47/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8951 - auc: 0.9609 - loss: 0.2473 - val_accuracy: 0.8886 - val_auc: 0.0000e+00 - val_loss: 0.3564 -
Epoch 48/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9067 - auc: 0.9628 - loss: 0.2401 - val_accuracy: 0.8928 - val_auc: 0.0000e+00 - val_loss: 0.3378 -
Epoch 49/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9152 - auc: 0.9706 - loss: 0.2187 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3525 -
Epoch 50/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8875 - auc: 0.9546 - loss: 0.2678 - val_accuracy: 0.8815 - val_auc: 0.0000e+00 - val_loss: 0.3701 -
Epoch 51/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9138 - auc: 0.9707 - loss: 0.2212 - val_accuracy: 0.8801 - val_auc: 0.0000e+00 - val_loss: 0.3693 -
Epoch 52/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9098 - auc: 0.9667 - loss: 0.2311 - val_accuracy: 0.8886 - val_auc: 0.0000e+00 - val_loss: 0.3716 -
Epoch 53/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9106 - auc: 0.9676 - loss: 0.2265 - val_accuracy: 0.8914 - val_auc: 0.0000e+00 - val_loss: 0.3656 -
Epoch 54/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8953 - auc: 0.9520 - loss: 0.2730 - val_accuracy: 0.8942 - val_auc: 0.0000e+00 - val_loss: 0.3399 -
Epoch 55/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9087 - auc: 0.9665 - loss: 0.2268 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3621 -
Epoch 56/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9040 - auc: 0.9541 - loss: 0.2713 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3417 -
Epoch 57/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9059 - auc: 0.9633 - loss: 0.2418 - val_accuracy: 0.8886 - val_auc: 0.0000e+00 - val_loss: 0.3493 -
Epoch 58/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9014 - auc: 0.9626 - loss: 0.2481 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3523 -
Epoch 59/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9071 - auc: 0.9637 - loss: 0.2416 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3537 -
Epoch 60/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9043 - auc: 0.9632 - loss: 0.2430 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3649 -
Epoch 61/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9053 - auc: 0.9619 - loss: 0.2548 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3666 -
Epoch 62/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9127 - auc: 0.9687 - loss: 0.2219 - val_accuracy: 0.8886 - val_auc: 0.0000e+00 - val_loss: 0.3569 -
Epoch 63/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9164 - auc: 0.9706 - loss: 0.2181 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3647 -
Epoch 64/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - accuracy: 0.9236 - auc: 0.9701 - loss: 0.2152 - val_accuracy: 0.8801 - val_auc: 0.0000e+00 - val_loss: 0.3685 -
Epoch 65/100
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8984 - auc: 0.9577 - loss: 0.2607 - val_accuracy: 0.8872 - val_auc: 0.0000e+00 - val_loss: 0.3616 -
Epoch 66/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9156 - auc: 0.9720 - loss: 0.2129 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3555 -
Epoch 67/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9078 - auc: 0.9658 - loss: 0.2337 - val_accuracy: 0.8956 - val_auc: 0.0000e+00 - val_loss: 0.3562 -
Epoch 68/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8982 - auc: 0.9615 - loss: 0.2484 - val_accuracy: 0.8914 - val_auc: 0.0000e+00 - val_loss: 0.3573 -
Epoch 69/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9207 - auc: 0.9720 - loss: 0.2130 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3631 -
Epoch 70/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9179 - auc: 0.9703 - loss: 0.2154 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3688 -
Epoch 71/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9076 - auc: 0.9642 - loss: 0.2337 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3661 -
Epoch 72/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9115 - auc: 0.9667 - loss: 0.2264 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3580 -
Epoch 73/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9067 - auc: 0.9693 - loss: 0.2182 - val_accuracy: 0.8815 - val_auc: 0.0000e+00 - val_loss: 0.3694 -
Epoch 74/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9097 - auc: 0.9690 - loss: 0.2275 - val_accuracy: 0.8787 - val_auc: 0.0000e+00 - val_loss: 0.3655 -
Epoch 75/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9174 - auc: 0.9667 - loss: 0.2296 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3666 -
Epoch 76/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9080 - auc: 0.9626 - loss: 0.2408 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3582 -
Epoch 77/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9099 - auc: 0.9671 - loss: 0.2246 - val_accuracy: 0.8872 - val_auc: 0.0000e+00 - val_loss: 0.3645 -
Epoch 78/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9075 - auc: 0.9645 - loss: 0.2374 - val_accuracy: 0.8801 - val_auc: 0.0000e+00 - val_loss: 0.3739 -
Epoch 79/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9174 - auc: 0.9700 - loss: 0.2167 - val_accuracy: 0.8872 - val_auc: 0.0000e+00 - val_loss: 0.3671 -
Epoch 80/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9001 - auc: 0.9666 - loss: 0.2295 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3731 -
Epoch 81/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9066 - auc: 0.9629 - loss: 0.2512 - val_accuracy: 0.8886 - val_auc: 0.0000e+00 - val_loss: 0.3600 -
```

```
from sklearn.metrics import precision_score, recall_score
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
test_loss, test_accuracy, test_auc = model.evaluate(X_test, y_test, verbose=0)
test_precision = precision_score(y_test, y_pred_classes)
test_recall = recall_score(y_test, y_pred_classes)

print(f"- Loss: {test_loss:.4f}")
print(f"- AUC: {test_auc:.4f}")
print(f"- Accuracy: {test_accuracy:.4f}")
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
- Loss: 0.2661
- AUC: 0.9554
- Accuracy: 0.9099
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_classes))
```

```
              precision    recall  f1-score   support

           0       0.90      0.93      0.91       394
           1       0.92      0.89      0.91       394

    accuracy                           0.91       788
   macro avg       0.91      0.91      0.91       788
weighted avg       0.91      0.91      0.91       788
```

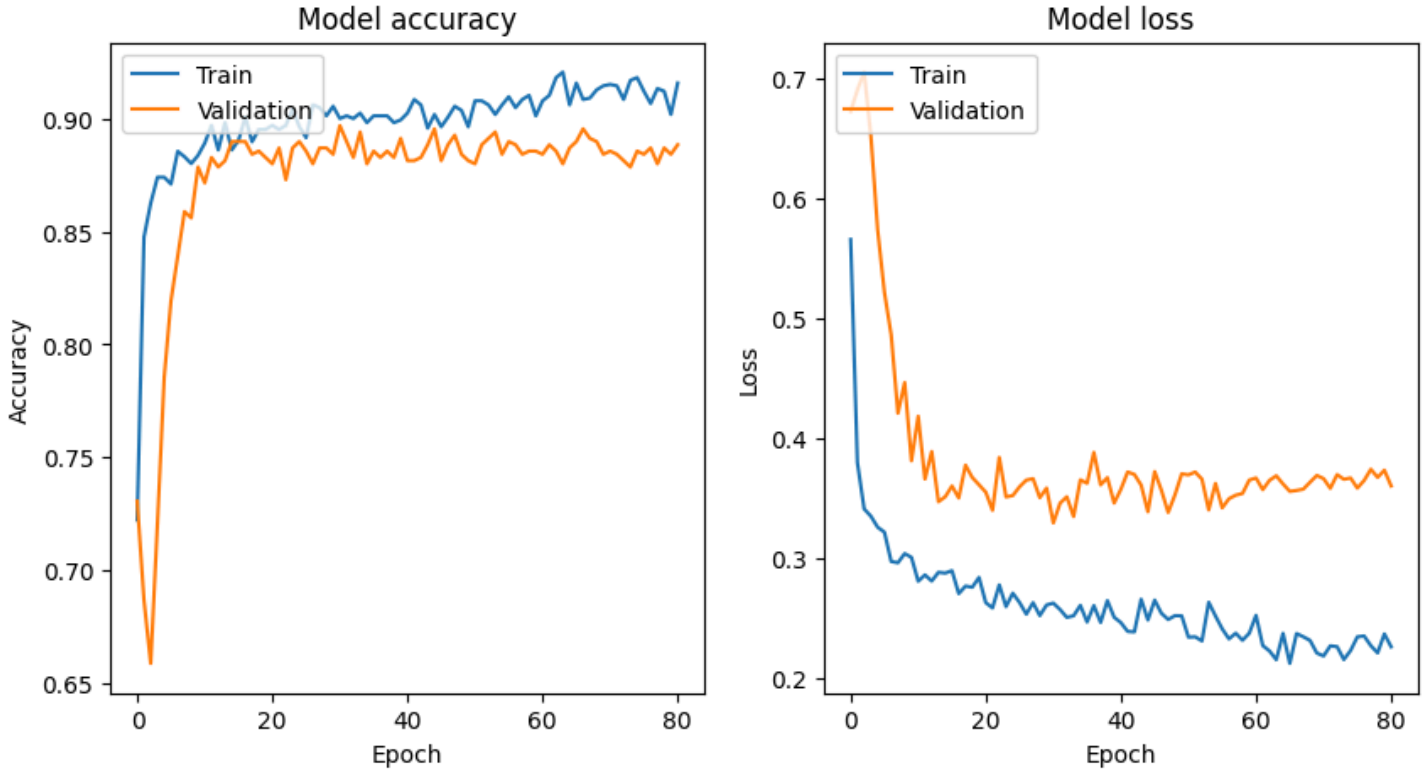
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
```

```
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.show()
```



PREDICT

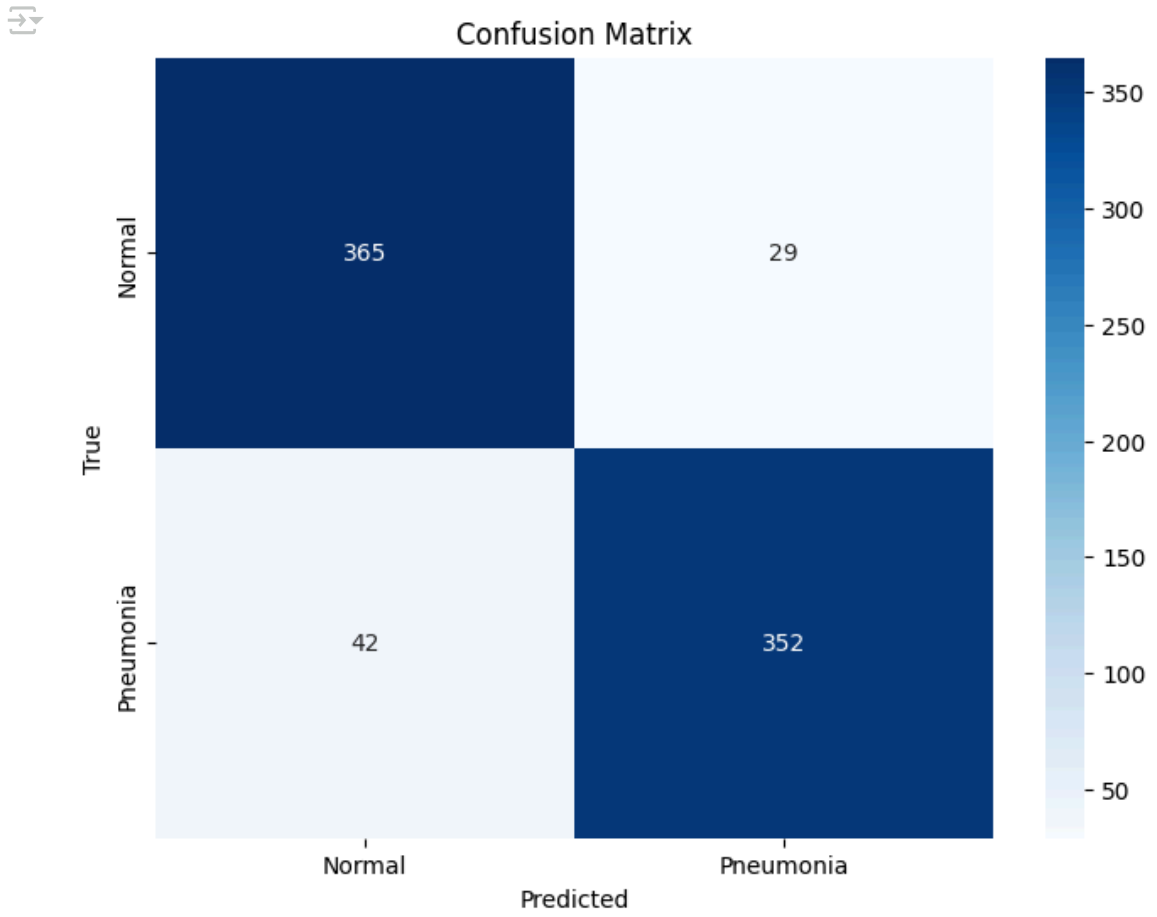
```
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

conf_matrix = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```



```
import matplotlib.pyplot as plt
import cv2
import pandas as pd
import os
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/NORMAL'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
predictions = []

for image_file in image_files:
    try:
        new_image = load_img(image_file, target_size=(150, 150))
        new_image_array = img_to_array(new_image)
        new_image_array = np.clip(new_image_array - 30, 0, 255)
        new_image_array = new_image_array / 255.0
        new_image_glcm = extract_glcm_features([new_image_array])
        new_image_glcm_df = pd.DataFrame(new_image_glcm)
        new_image_scaled = scaler.transform(new_image_glcm_df)
        prediction = model.predict(new_image_scaled)
        predicted_class = (prediction > 0.5).astype(int)[0][0]

        image_paths.append(image_file)
        predictions.append('Pneumonia' if predicted_class == 1 else 'Normal')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

results_df = pd.DataFrame({'Image Path': image_paths, 'Prediction': predictions})
print(results_df)

sample_files = random.sample(image_files, 5)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))
fig.suptitle('Sample Predictions', fontsize=16)

for i, image_file in enumerate(sample_files):
    try:
        img = cv2.imread(image_file, cv2.IMREAD_GRAYSCALE)

        axes[i].imshow(img, cmap='gray')
        prediction_for_sample = results_df[results_df['Image Path'] == image_file]['Prediction'].values[0]
```

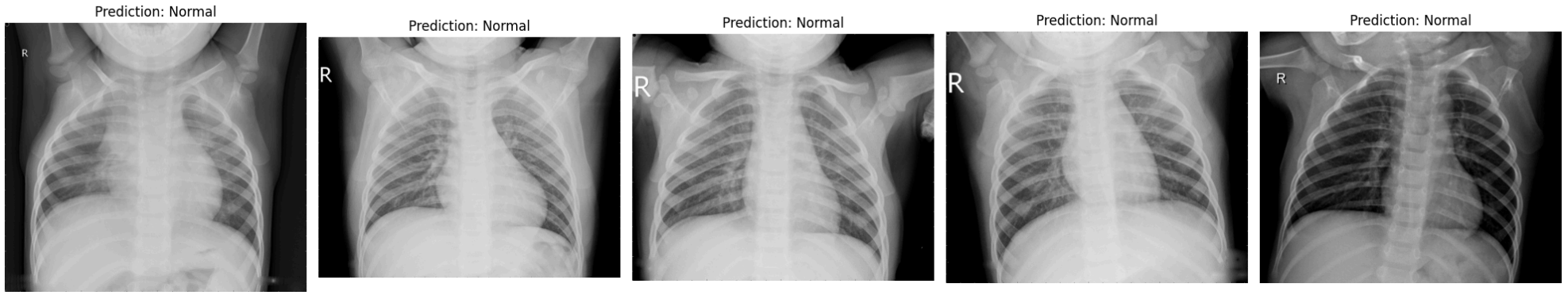
```
            axes[i].set_title(f"Prediction: {prediction_for_sample}")
            axes[i].axis('off')

        except Exception as e:
            print(f"Error processing {image_file}: {e}")

    plt.tight_layout()
    plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
```

	Image Path	Prediction
0	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
1	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
2	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
3	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
4	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
5	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
6	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
7	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal




```
import matplotlib.pyplot as plt
import cv2
import pandas as pd
import os
```

```
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/PNEUMONIA'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
predictions = []

for image_file in image_files:
    try:
        new_image = load_img(image_file, target_size=(150, 150))
        new_image_array = img_to_array(new_image)
        new_image_array = np.clip(new_image_array - 30, 0, 255)
        new_image_array = new_image_array / 255.0
        new_image_glcm = extract_glcm_features([new_image_array])
        new_image_glcm_df = pd.DataFrame(new_image_glcm)
        new_image_scaled = scaler.transform(new_image_glcm_df)
        prediction = model.predict(new_image_scaled)
        predicted_class = (prediction > 0.5).astype(int)[0][0]

        image_paths.append(image_file)
        predictions.append('Pneumonia' if predicted_class == 1 else 'Normal')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

results_df = pd.DataFrame({'Image Path': image_paths, 'Prediction': predictions})
print(results_df)

sample_files = random.sample(image_files, 5)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))
fig.suptitle('Sample Predictions', fontsize=16)

for i, image_file in enumerate(sample_files):
    try:
        img = cv2.imread(image_file, cv2.IMREAD_GRAYSCALE)

        axes[i].imshow(img, cmap='gray')
        prediction_for_sample = results_df[results_df['Image Path'] == image_file]['Prediction'].values[0]
        axes[i].set_title(f"Prediction: {prediction_for_sample}")
        axes[i].axis('off')

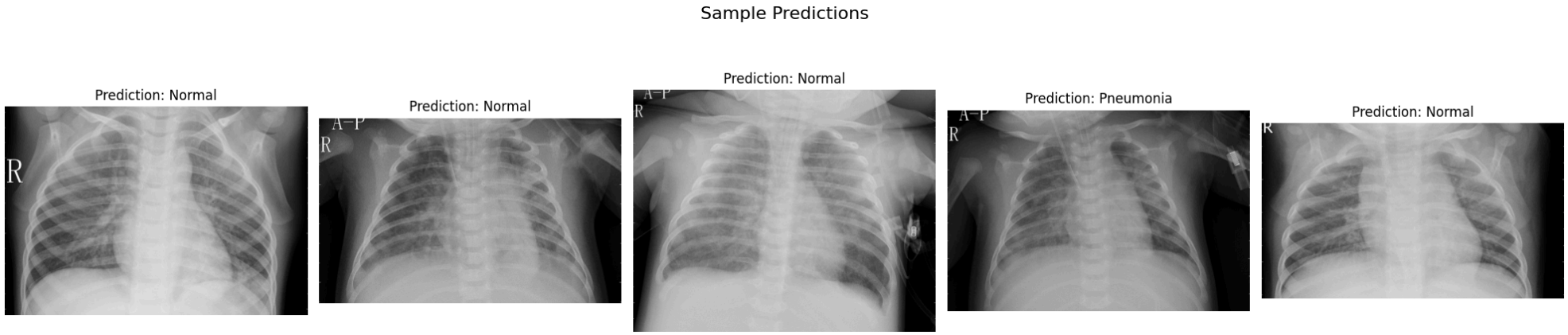
    except Exception as e:
```

```
        print(f"Error processing {image_file}: {e}")

plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
```

	Image Path	Prediction
0	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
1	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
2	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
3	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
4	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
5	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
6	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Pneumonia
7	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Pneumonia




```
# prompt: uji data glcm ini juga dengan svm

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create and train the SVM model
svm_model = SVC(kernel='linear', random_state=42) # You can choose different kernels
svm_model.fit(X_train, y_train)
```

```
# Make predictions on the test set
y_pred_svm = svm_model.predict(X_test)

# Evaluate the SVM model
print("\n--- SVM Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_svm))

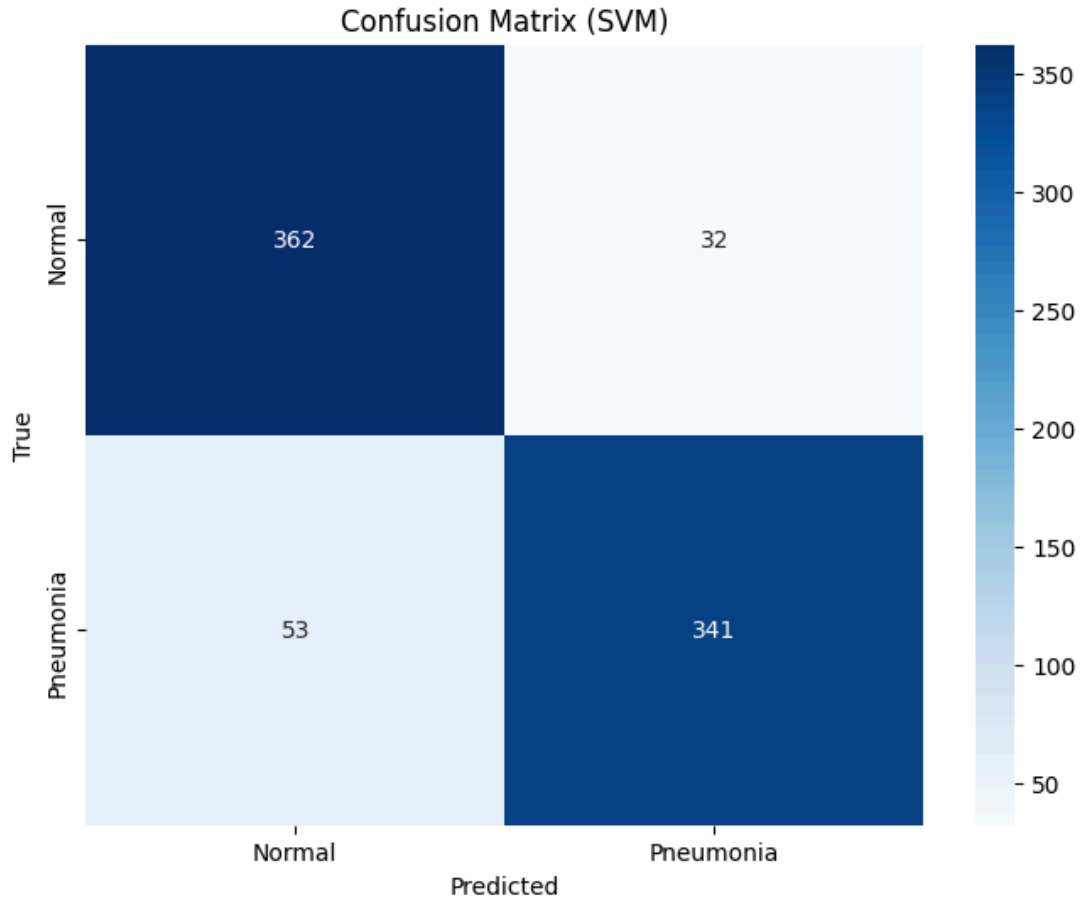
# Confusion Matrix
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (SVM)')
plt.show()
```

```
--- SVM Model Evaluation ---
Accuracy: 0.8921

Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.92      0.89       394
           1       0.91      0.87      0.89       394

    accuracy                           0.89       788
   macro avg       0.89      0.89      0.89       788
weighted avg       0.89      0.89      0.89       788
```



```
# prompt: uji juga dengan random forest

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Create and train the Random Forest model
# You can adjust n_estimators (number of trees) and other parameters
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_rf = rf_model.predict(X_test)

# Evaluate the Random Forest model
print("\n--- Random Forest Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_rf):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))

# Confusion Matrix
conf_matrix_rf = confusion_matrix(y_test, y_pred_rf)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_rf, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (Random Forest)')
plt.show()
```